

개정판
으뜸 파이썬



12장 넘파이

학습목표

- 과학 계산에 필수적인 넘파이 라이브러리의 용도와 사용법에 대해 이해한다.
- 넘파이의 핵심적인 요소인 ndarray를 사용해 본다.
- ndarray와 리스트의 차이를 이해하고 그 특성에 맞는 활용을 할 수 있다.
- 넘파이가 과학기술 분야와 머신러닝 분야에 적합한 이유를 설명할 수 있다.
- ndarray와 선형대수 문제의 연관을 이해한다.
- 넘파이를 이용하여 다양한 행렬, 벡터 문제를 해결할 수 있다.
- 넘파이의 linalg 패키지를 이용하여 다양한 선형대수 문제를 풀 수 있다.
- 파이썬의 리스트와 넘파이의 ndarray를 연동하여 계산할 수 있는 기초 지식을 습득한다.



12.1 넘파이 라이브러리

- 넘파이 NumPy

- 파이썬의 과학계산을 위한 가장 기본적인 라이브러리
- pip를 이용하여 명령행에서 다음과 같이 입력한다.

```
pip install numpy
```

- 아나콘다, 미니콘다라는 패키지를 설치하면 자동 설치가 됨
- 구글 colab 환경에서는 설치가 필요 없음

- 특징

- 강력한 다차원 배열 처리 : ndarray 제공, 효과적 다차원 데이터 처리 가능
- 빠른 연산 속도 : 내부적으로 C 언어로 구현, 파이썬 내장 리스트에 비해 빠른 연산 속도, 대량 데이터 처리 시 효율성 극대화
- 다양한 수학 함수 제공 : 간단한 사칙연산부터 난수 생성 등과 같은 복잡한 수학함수, 다양한 함수를 제공
- 편리한 브로드캐스팅 기능 : 코드를 직관적이고도 간결하게 만들어 줌
- 다른 라이브러리와의 호환성 : 판다스, 맷플롯립, 사이킷런, 텐서플로 등 다른 많은 라이브러리들이 넘파이 배열을 기반으로 동작



12.1 넘파이 라이브러리

설치와 테스트

명령 프롬프트에서 설치 명령어
pip는 파이썬 패키지 관리 프로그램임

```
C:\workspace\file_work>pip install numpy
Collecting numpy
  Obtaining dependency information for numpy from https://files.pythonhosted.org/packages/72/b2/02770e60c4e2f7e158d923ab0dea4e9f146a2dbf267fec6d8dc61d475689/numpy-1.25.2-cp311-cp311-win_amd64.whl.metadata
    Downloading numpy-1.25.2-cp311-cp311-win_amd64.whl.metadata (5.7 kB)
    Downloading numpy-1.25.2-cp311-cp311-win_amd64.whl (15.5 MB)
      15.5/15.5 MB 7.3 MB/s eta 0:00:00
Installing collected packages: numpy
Successfully installed numpy-1.25.2

C:\workspace\file_work>python
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun 7 2023, 05:45:37) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more
>>> import numpy
>>> numpy.random.rand(2,2)
array([[0.92071858, 0.31389027],
       [0.90846526, 0.51377693]])
>>>
```

1) pip 명령으로 넘파이 패키지를 설치함

2) 넘파이 패키지 테스트 코드 실행

간단한 테스트 코드로 2x2 크기의 랜덤 행렬을 생성함

[그림 12-1] pip를 이용한 numpy 설치 화면과 간단한 테스트 코드 실행 화면



12.1 넘파이 라이브러리



설치와 테스트



대화창 실습: numpy의 ndarray 사용

numpy의 별칭은 np

```
>>> import numpy as np
>>> a = np.array([1, 2, 3]) # 넘파이 ndarray 객체의 생성
>>> a
array([1, 2, 3])
>>> a.shape                # a 객체의 형태(shape)
(3,)
>>> a.ndim                 # a 객체의 차원
1
>>> a.dtype                # a 객체 내부 자료형
dtype('int32')
>>> a.itemsize             # a 객체 내부 자료형이 차지하는 메모리 크기(byte)
4
>>> a.size                 # a 객체의 전체 크기(항목의 수)
3
```



12.1 넘파이 라이브러리



ndarray

- 넘파이의 가장 핵심적인 객체, 다차원 배열을 처리하며 다음과 같은 속성들이 있다.

```
a = np.array([1, 2, 3])
```

1	2	3
---	---	---

```
a.shape : (3,)
a.ndim : 1
a.dtype : dtype('int32')
a.size : 3
a.itemsize : 4
```

shape은 생성된 배열 객체의 형태를 튜플 타입으로 반환함

[그림 12-2] 넘파이의 ndarray a와 그 속성들



12.1 넘파이 라이브러리



대화창 실습: 리스트 자료형의 덧셈과 나눗셈

```
>>> student_mid = [70, 85, 90, 75] # 중간시험 성적
>>> student_fin = [90, 65, 70, 85] # 기말시험 성적
>>> student_sum = student_mid + student_fin
>>> student_sum # 학생들의 중간시험 성적과 기말시험 성적의 합
[70, 85, 90, 75, 90, 65, 70, 85]
>>> student_sum / 2 # 오류! : 리스트는 나누기 연산을 지원하지 않음
student_sum / 2
TypeError: unsupported operand type(s) for /: 'list' and 'int'
```



대화창 실습: 넘파이 다차원 배열의 덧셈과 나눗셈

```
>>> student_mid = np.array([70, 85, 90, 75]) # 중간시험 성적
>>> student_fin = np.array([90, 65, 70, 85]) # 기말시험 성적
>>> student_sum = student_mid + student_fin
>>> student_sum # 학생들의 중간시험 성적과 기말시험 성적의 합
array([160, 150, 160, 160])
>>> student_sum / 2 # 넘파이 배열은 나누기 연산을 지원함
array([80., 75., 80., 80.]
```

- 리스트는 나누기 연산 지원 안 함
-> 넘파이의 다차원 배열로 고쳐서 수행

- **벡터화**vectorization 연산
- 원소끼리의 연산이 이루어지는 것
- **브로드캐스팅**broadcasting
- 모든 원소에 대해 같은 연산을 적용



12.1 넘파이 라이브러리



주의 - ndarray 배열을 생성할 때 주의할 점

1. 넘파이의 배열 ndarray를 생성할 때, 반드시 대괄호를 사용하여 리스트나 튜플과 같은 시퀀스 자료형을 만들어서 array() 함수의 인자로 넣어야 한다.

```
>>> a = np.array([1, 2, 3, 4]) # 리스트로부터 다차원 배열 생성
>>> b = np.array((1, 2, 3, 4)) # 튜플로부터 다차원 배열 생성
>>> c = np.array(range(4))      # range()로부터 다차원 배열 생성
```

만일 단순히 여러 데이터를 쉼표로 구분해서 입력할 경우 오류가 발생한다.

```
>>> a = np.array(1, 2, 3, 4) # 잘못된 입력
...
ValueError: only 2 non-keyword arguments accepted
```




12.1 넘파이 라이브러리

Welcome to Python



2. ndarray는 dtype으로 자료형을 지정할 경우에 다른 자료형의 값을 원소로 가질 수 없다.

```
>>> a = np.array([1, 'two', 3, 4], dtype = np.int32)
...
ValueError: invalid literal for int() with base 10: 'two'
```

만일 다음과 같이 문자열이 포함되어 있고 별도의 자료형을 명시하지 않을 경우 'two'라는 원소의 자료형인 str 형으로 자동 형 변환이 일어난다. 이 경우 모든 원소들은 문자열 형이 되어 정수의 덧셈, 뺄셈 등의 연산을 사용할 수 없다.

```
>>> a = np.array([1, 'two', 3, 4]) # 자료형은 문자열 형이 됨
>>> a
array(['1', 'two', '3', '4'], dtype = '<U21')
```



12.1 넘파이 라이브러리

노트



NOTE: 넘파이 배열의 데이터 타입을 지정하는 두 가지 방법

넘파이 배열의 데이터 타입을 지정하는 방법에는 두 가지가 있다.

1. `dtype = np.int32`와 같이 넘파이의 `int32` 속성 값으로 지정하기

```
>>> a = np.array([1, 2, 3, 4], dtype = np.int32)
```

2. `dtype = 'int32'`와 같이 문자열 형식으로 속성 값 지정하기

```
>>> a = np.array([1, 2, 3, 4], dtype = 'int32')
```



12.1 넘파이 라이브러리

- numpy의 ndarray의 속성

[표 12-1] 넘파이에 있는 ndarray의 속성들

속성	설명
ndim	배열 축 혹은 차원의 개수
shape	배열의 차원으로 (m, n) 형식의 튜플 형이다. 이때, m과 n은 각 차원의 크기를 알려주는 정수값이다.
size	배열의 원소의 개수이다. 이 개수는 shape 내의 원소의 크기의 곱과 같다. 즉, (m, n) shape 배열의 size는 m*n이다.
dtype	배열 내의 원소의 형을 기술하는 객체이다. 넘파이는 파이썬 표준 자료형을 사용할 수 있으나, 넘파이 자체의 자료형인 bool_, character, int_, int8, int16, int32, int64, float, float8, float16_, float32, float64, complex_, complex64, object_ 형을 사용할 수 있다.
itemsize	배열 내의 원소의 크기를 바이트 단위로 기술한다. 예를 들어, int32 자료형의 크기는 32/8 = 4바이트가 된다.
data	배열의 실제 원소를 포함하고 있는 버퍼



12.1 넘파이 라이브러리



LAB 12-1: ndarray 객체 생성하기 그리고 속성 알아보기

1. 0에서 9까지의 정수값을 가지는 ndarray 객체 a를 넘파이를 이용하여 작성하여 다음과 같이 출력하여라.

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

2. range() 함수를 사용하여 0에서 9까지의 정수값을 가지는 ndarray 객체 b를 만들고 문제 1의 결과와 같이 나타나도록 하여라.



12.1 넘파이 라이브러리

Welcome to Python



3. 문제 2의 코드를 수정하여 0에서 9까지의 정수값 중에서 다음과 같이 짝수를 가지는 ndarray 객체 `c`를 출력하여라.

```
array([0, 2, 4, 6, 8])
```

4. 문제 3번 ndarray 객체 `c`의 `shape`, `ndim`, `dtype`, `size`, `itemsize`를 다음과 같이 출력하여라.

```
c.shape = (5,)
c.ndim = 1
c.dtype = int64
c.size = 5
c.itemsize = 8
```



12.2 ndarray의 메소드와 주요 함수



ndarray

대화창 실습: ndarray의 메소드

```
>>> a = np.array([10,20,30]) # 1차원 ndarray 배열 생성
>>> a.max()                # 가장 큰 값을 반환
30
>>> a.min()                # 가장 작은 값을 반환
10
>>> a.mean()               # 평균 값을 반환
20.0
```

flatten()

대화창 실습: ndarray의 flatten() 메소드

```
>>> a = np.array([[1, 1], [2, 2], [3, 3]])
>>> a.flatten()            # ndarray 배열을 1차원 배열로 만드는 평탄화 메소드
array([1, 1, 2, 2, 3, 3])
```



12.2 ndarray의 메소드와 주요 함수



append() 함수

대화창 실습: ndarray의 append() 함수

```
>>> a = np.array([1, 2, 3])
>>> b = np.array([[4, 5, 6], [7, 8, 9]])
>>> np.append(a, b)
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> np.append([a], b, axis = 0) # [a]를 통해 2차원 배열로 만들어야 함
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

rand() 함수

대화창 실습: ndarray의 rand() 함수

```
>>> np.random.rand(3, 3) # (3, 3) shape의 난수 생성
array([[0.63208882, 0.72259476, 0.15125742],
       [0.60731818, 0.20682056, 0.51958311],
       [0.644331 , 0.91940484, 0.83990604]])
```



12.2 ndarray의 메소드와 주요 함수



randint() 함수



대화창 실습: ndarray의 randint() 함수

```
>>> np.random.randint(0, 10, size = 10) # 0에서 10 사이의 10개의 난수 생성  
array([2, 0, 8, 2, 7, 0, 1, 3, 1, 3])
```




12.2 ndarray의 메소드와 주요 함수



LAB 12-2: ndarray 객체의 메소드와 함수

1. [23, 45, 67, 7, 2, 30, 34, 82]의 정수 값을 가지는 ndarray 객체 a를 생성하여 다음과 같이 출력하여라.

```
a = array([23 45 67 7 2 30 34 82])
```

이 ndarray의 max(), min(), mean() 메소드를 이용하여 최댓값, 최솟값, 평균을 다음과 같이 출력하여라.

최댓값 : 82

최솟값 : 2

평균 : 36.25



12.2 ndarray의 메소드와 주요 함수

2. `numpy.random.randint()` 함수를 사용하여 0에서 99까지의 정수값 10개를 랜덤하게 생성하시오. 이 10개의 값을 `b`라는 ndarray에 넣고 ndarray에서 최댓값, 최솟값, 평균을 다음과 같이 출력하여라(`b` 값은 랜덤하게 생성되므로 다음 화면의 값과 일치하지 않음).

```
b = [25 2 9 86 93 73 15 53 67 20]
```

최댓값 : 93

최솟값 : 2

평균 : 44.3

3. 위의 문제 1의 `a` 배열과 문제 2의 `b` 배열을 `append()`하여 다음과 같은 배열 `c`를 생성하여 출력하시오.

```
c = [23 45 67 7 2 30 34 82, 25 2 9 86 93 73 15 53 67 20]
```



12.3 ndarray의 연산



대화창 실습: ndarray의 덧셈(shape이 같을 경우)

```
>>> a = np.array([1, 2, 3])      # 1, 2, 3 원소를 가지는 1차원 ndarray
>>> b = np.array([4, 5, 6])      # 4, 5, 6 원소를 가지는 1차원 ndarray
>>> c = a + b                    # 1차원 ndarray의 덧셈
>>> c
array([5, 7, 9])
```



대화창 실습: ndarray의 연산

```
>>> sal = np.array([300, 360, 400, 380]) # 네 명의 급여
>>> sal_w_bonus = sal + 100              # 네 명의 급여 각각에 100을 더함
>>> sal_inc = sal * 1.2                  # 네 명의 급여 각각에 1.2를 곱함
>>> print('100만 원 보너스 추가 후:', sal_w_bonus)
100만 원 보너스 추가 후: [400 460 500 480]
>>> print('20% 인상된 급여:', sal_inc)
20% 인상된 급여: [360. 432. 480. 456.]
```

1	2	3
+		
4	5	6
=		
5	7	9



12.3 ndarray의 연산



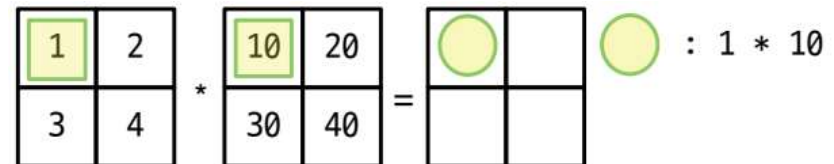
ndarray의 사칙연산



대화창 실습: 2차원 ndarray의 사칙연산

```
>>> a = np.array([[1, 2], [3, 4]]) # 2차원 배열 a
>>> b = np.array([[10, 20], [30, 40]]) # 2차원 배열 b
>>> a + b
array([[11, 22],
       [33, 44]])
>>> a - b
array([[ -9, -18],
       [-27, -36]])
>>> a * b
array([[ 10, 40],
       [ 90, 160]])
>>> a / b
array([[0.1, 0.1],
       [0.1, 0.1]])
```

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} = \begin{bmatrix} 1 \times 10 & 2 \times 20 \\ 3 \times 30 & 4 \times 40 \end{bmatrix}$$



[그림 12-4] 넘파이의 $a * b$ 연산



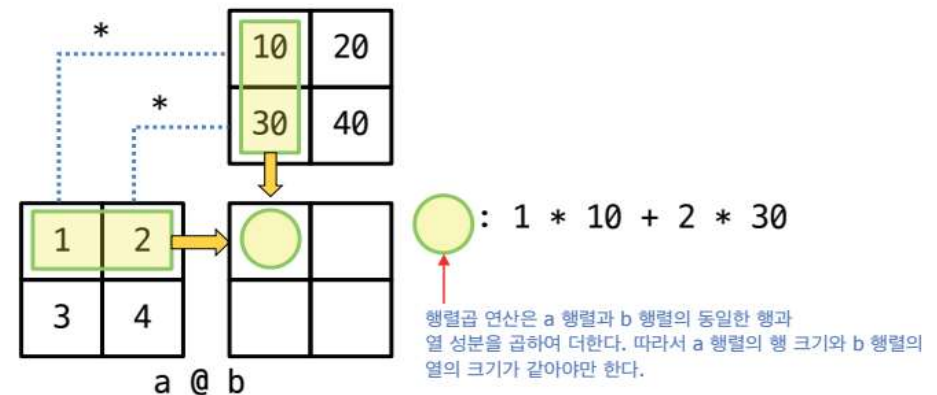
12.3 ndarray의 연산

행렬 곱 matmul()

대화창 실습: 행렬 곱 함수 matmul() 실습

```
>>> np.matmul(a, b)
array([[ 70, 100],
       [150, 220]])
>>> a @ b # 행렬 곱 연산자 @를 사용해 보자.
array([[ 70, 100],
       [150, 220]])
>>> a[0,0] * b[0,0] + a[0,1] * b[1,0] # 1행 1열 성분이 계산되는 과정
70
>>> a[0,0] * b[0,1] + a[0,1] * b[1,1] # 1행 2열 성분이 계산되는 과정
100
>>> a[1,0] * b[0,0] + a[1,1] * b[1,0] # 2행 1열 성분이 계산되는 과정
150
>>> a[1,0] * b[0,1] + a[1,1] * b[1,1] # 2행 2열 성분이 계산되는 과정
220
```

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} @ \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} = \begin{bmatrix} 1 \times 10 + 2 \times 30 & 1 \times 20 + 2 \times 40 \\ 3 \times 10 + 4 \times 40 & 3 \times 20 + 4 \times 40 \end{bmatrix}$$



[그림 12-5] 넘파이의 a @ b 연산



12.3 ndarray의 연산



행렬 곱 matmul()



대화창 실습: 단위행렬에 대한 행렬 곱

```
>>> a = [[1, 2], [3, 4]]
>>> b = [[1, 0], [0, 1]] # b는 단위행렬
>>> np.matmul(a, b)      # a 행렬과 단위행렬 b의 곱의 결과
array([[ 1,  2],
       [ 3,  4]])
```

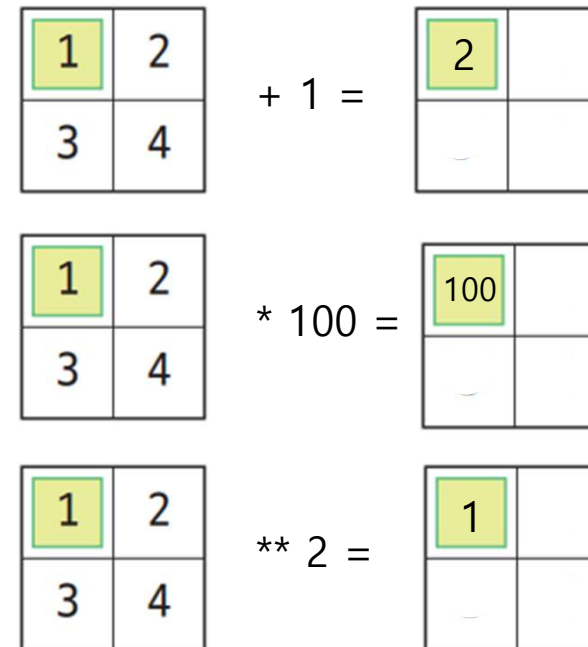
$$\begin{aligned} AE &= \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} a \times 1 + b \times 0 & a \times 0 + b \times 1 \\ c \times 1 + d \times 0 & c \times 0 + d \times 1 \end{pmatrix} \\ &= \begin{pmatrix} a & b \\ c & d \end{pmatrix} \end{aligned}$$



12.3 ndarray의 연산

 대화창 실습: 2차원 ndarray의 덧셈, 뺄셈, 곱셈, 나눗셈, 제곱 연산

```
>>> a = np.array([[1, 2], [3, 4]])
>>> a + 1    # 행렬의 각 성분에 대한 덧셈
array([[2, 3],
       [4, 5]])
>>> a - 1    # 행렬의 각 성분에 대한 뺄셈
array([[0, 1],
       [2, 3]])
>>> a * 100  # 행렬의 각 성분에 대한 곱셈
array([[100, 200],
       [300, 400]])
>>> a / 100  # 행렬의 각 성분에 대한 나눗셈
array([[0.01, 0.02],
       [0.03, 0.04]])
>>> a ** 2   # 행렬의 각 성분에 대한 제곱 연산
array([[ 1,  4],
       [ 9, 16]])
```





12.3 ndarray의 연산



LAB 12-3: 다차원 행렬의 생성과 활용

1. 1에서 9까지의 모든 정수값을 크기 순서대로 가지는 3x3 크기의 행렬 **a**를 생성하고 모든 성분의 값이 2인 3x3 크기의 행렬 **b**를 생성하여라.

```
a = [[1 2 3]
      [4 5 6]
      [7 8 9]]
b = [[2 2 2]
      [2 2 2]
      [2 2 2]]
```

2. 다음 행렬 연산의 결과를 예상한 후 실행하고 그 결과를 적으시오.

- 1) $a + b$
- 2) $a - b$
- 3) $a * b$
- 4) a / b
- 5) $a @ b$
- 6) $a ** 2$

이 ndarray의 `max()`, `min()`, `mean()` 메소드를 이용하여 최댓값, 최솟값, 평균을 다음과 같이 출력하여라.

```
최댓값 : 82
최솟값 : 2
평균 : 36.25
```




12.3 ndarray의 연산



3. `numpy.random.randint()` 함수를 사용하여 0에서 99까지의 정수 값 10개를 랜덤하게 생성하시오. 이 10개의 값을 `b`라는 `ndarray`에 넣고 `ndarray`에서 최댓값, 최솟값, 평균을 다음과 같이 출력하여라(`b` 값은 랜덤하게 생성되므로 다음 화면의 값과 일치하지 않음).

```
b = [25 2 9 86 93 73 15 53 67 20]
```

최댓값 : 93

최솟값 : 2

평균 : 44.3

4. 위의 문제 1의 `a` 배열과 문제 2의 `b` 배열을 `append()`하여 다음과 같은 배열 `c`를 생성하여 출력하시오.

```
c = [23 45 67 7 2 30 34 82, 25 2 9 86 93 73 15 53 67 20]
```



12.3 ndarray의 연산

Welcome to Python



주의 - 넘파이 ndarray 초기화 함수의 호출

`zeros((n, m))` 함수를 호출할 때, `zeros(n, m)`으로 호출하지 않도록 한다. 만일 `np.zeros(3, 4)`와 같이 호출하면 데이터 형을 인식할 수 없다는 오류가 나타난다.

`TypeError: data type not understood`

`ones()`, `full()`, `random.random()` 함수를 호출할 때도 이중 괄호 내에 생성할 행렬의 크기를 지정해 주도록 한다.



12.4 다양한 행렬의 생성



- 넘파이는 배열을 쉽게 생성하는 함수도 지원하고 있다.
- 첫 번째가 `zeros((n,m))` 함수이다. 이는 $n \times m$ 배열(혹은 행렬)을 생성해서 초기값은 0으로 해준다.
- `ones((n,m))` 함수를 이용하면 초기값을 1로 가지는 $n \times m$ 배열을 생성할 수 있다.
- `full((n,m), x)` 함수를 이용하면 $n \times m$ 크기의 초기값이 x 인 행렬을 생성할 수 있다.
- `eye(n)` 함수를 이용하면 $n \times n$ 크기의 단위행렬(identity matrix)을 생성할 수 있다.
 - 정사각행렬(square matrix) 중에서 대각선 성분이 1이고 나머지 성분이 0인 행렬



대화창 실습: 파이썬 표현식의 사용

```
>>> np.zeros((2, 3))
array([[0., 0., 0.],
       [0., 0., 0.]])
```

모든 값이 0인 2x3 크기 행렬

```
>>> np.ones((2, 3))
array([[1., 1., 1.],
       [1., 1., 1.]])
```

모든 값이 1인 2x3 크기 행렬

```
>>> np.full((2, 3), 100)
array([[100, 100, 100],
       [100, 100, 100]])
```

모든 값이 100인 2x3 행렬



12.4 다양한 행렬의 생성



대화창 실습: 파이썬 표현식의 사용

```
>>> np.zeros((2, 3))      # 2행 3열 크기의 영행렬을 생성함
array([[0., 0., 0.],
       [0., 0., 0.]])

>>> np.ones((2, 3))      # 2행 3열 크기의 1 값을 가지는 행렬을 생성함
array([[1., 1., 1.],
       [1., 1., 1.]])

>>> np.full((2, 3), 100)  # 2행 3열 크기의 100 값을 가지는 행렬을 생성함
array([[100, 100, 100],
       [100, 100, 100]])

>>> np.eye(3)             # 3행 3열 크기의 단위 행렬을 생성함
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])

>>> np.random.random((2, 3)) # 2행 3열 난수값을 가지는 행렬을 생성함
array([[0.87143684, 0.44538612, 0.11908973],
       [0.87548557, 0.57502347, 0.87641631]])
```

3x3 크기의 단위행렬

2x3 크기의 랜덤행렬



12.4 다양한 행렬의 생성



`np.arange(0, 10)`

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

[그림 12-6] 넘파이의 `arange(0, 10)` 실행 결과

`np.arange(0, 10, 2)`

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

step=2

[그림 12-7] 넘파이의 `arange(0, 10, 2)` 실행과 스텝 값의 역할



대화창 실습: `arange()` 함수를 이용한 배열 생성

```
>>> np.arange(0, 10)
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> np.arange(0, 10, 2)
array([0, 2, 4, 6, 8])
>>> np.arange(0.0, 1.0, 0.2)
array([0., 0.2, 0.4, 0.6, 0.8])
```

실수 step 값을 사용할 수 있음



12.4 다양한 행렬의 생성

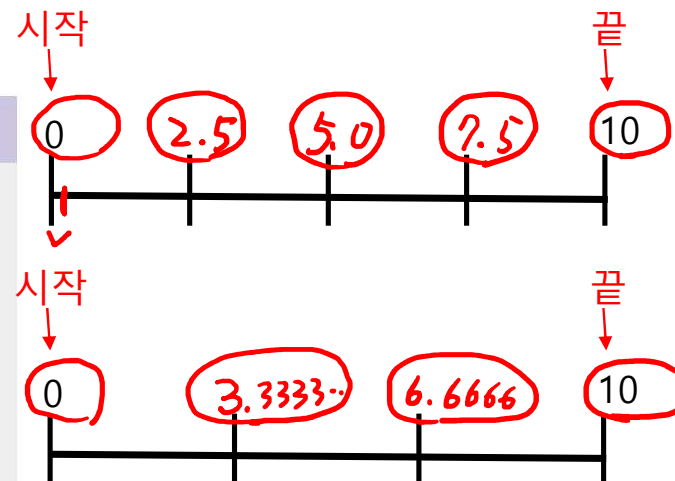
동일한 간격을 가진 연속된 값을 생성



대화창 실습: linspace() 함수를 이용한 배열 생성

```
>>> np.linspace(0, 10, 5) # 0에서 10 사이의 간격을 균등하게 나눈다.
array([ 0. , 2.5, 5. , 7.5, 10. ])
>>> np.linspace(0, 10, 4)
array([ 0. , 3.3333333, 6.6666667, 10. ])
>>> np.logspace(0, 5, 10) # 0에서 10 사이의 간격을 로그 스케일로 나눈다.
array([1.00000000e+00, 3.59381366e+00, 1.29154967e+01, 4.64158883e+01,
       1.66810054e+02, 5.99484250e+02, 2.15443469e+03, 7.74263683e+03,
       2.78255940e+04, 1.00000000e+05])
```

(시작, 끝, 간격의 수)를 인자로 가짐.
디폴트 간격의 수는 50개



데이터 생성을 시작할 값 - 생략할 수 없음

데이터 생성 개수 - 기본값은 50개

`np.linspace(start, stop, num=50)`

데이터 생성을 멈추기 위한 마지막 값으로 생략할 수 없음
데이터는 stop-1이 아니라 stop까지 생성된다.

정수가 아니라 실수 데이터가 생성된다.
start에서 stop까지의 간격을 균일하게 쪼개어 num 개의 실수를 생성한다

데이터 생성을 10^{start} 부터 시작한다.

데이터 생성 개수 - 기본값은 50개

`np.logspace(start, stop, num=50)`

데이터 생성을 멈추기 위한 값으로 생략할 수 없음.
데이터는 10^{stop-1}이 아니라 10^{stop}까지 생성된다.

10^{start} 부터 10^{stop} 까지의 실수를 로그 스케일로 볼 때 균등한 간격으로 num 개 생성한다.
여기서는 10을 밑으로 잡았지만, base 키워드 매개변수에 설정한 인자에 따라 바꿀 수도 있다.



12.4 다양한 행렬의 생성

Welcome to Python



LAB 12-4: 행렬의 생성

1. `full()` 함수를 이용하여 모든 원소의 값이 2인 3x3 크기의 행렬 `a1`을 생성하여라.

```
a1 = [[2 2 2]
       [2 2 2]
       [2 2 2]]
```

2. `arange()` 함수를 사용하여 1에서 12까지 12개의 원소를 가지는 1차원 행렬 `a2`를 생성하여라.

```
a2 = [ 1 2 3 4 5 6 7 8 9 10 11 12]
```

3. `arange()` 함수의 `step` 값을 이용하여 1에서 50까지 정수 중에서 3의 배수만을 가지는 행렬 `a3`를 생성하여라.

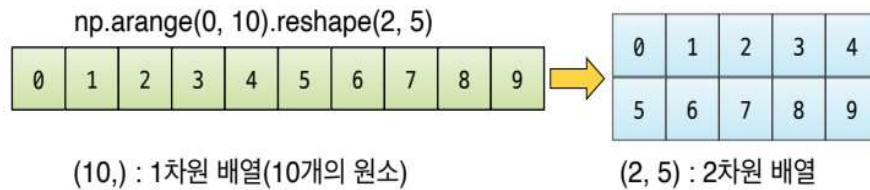
```
a3 = [ 3 6 9 12 15 18 21 24 27 30 33 36 39 42 45 48]
```

4. 0에서 20 사이에서 동일한 간격의 값을 가지는 원소 5개를 가진 `a4` 행렬을 생성하여라.

```
a4 = [ 0. 5. 10. 15. 20.]
```



12.5 ndarray의 재구성 `reshape`



[그림 12-9] `reshape()` 메소드를 이용하여 1차원 배열을 2행 5열의 다차원 행렬로 변환시킨 결과

대화창 실습: `reshape()`을 이용한 배열의 재구성

```
>>> np.arange(0, 10).reshape(2, 5)
```

```
array([[0, 1, 2, 3, 4],
       [5, 6, 7, 8, 9]])
```

```
>>> np.arange(0, 10).reshape(5, 2)
```

```
array([[0, 1],
       [2, 3],
       [4, 5],
       [6, 7],
       [8, 9]])
```

```
>>> np.arange(0, 10).reshape(3, 3)
```

```
...
```

```
ValueError: cannot reshape array of size 10 into shape (3, 3)
```




12.5 ndarray의 재구성reshape

대화창 실습: 다른 차원으로의 reshape() 실습

```
>>> np.arange(0, 24).reshape(4, 3, 2)
array([[[ 0, 1],
        [ 2, 3],
        [ 4, 5]],
       [[ 6, 7],
        [ 8, 9],
        [10, 11]],
       [[12, 13],
        [14, 15],
        [16, 17]],
       [[18, 19],
        [20, 21],
        [22, 23]]])
```

대화창 실습: transpose() 함수

```
>>> a = np.arange(6).reshape(3, 2) # 3행 2열 크기의 행렬을 만든다.
>>> a
array([[0, 1],
       [2, 3],
       [4, 5]])
>>> np.transpose(a) # a 행렬의 전치 행렬을 만든다.
array([[0, 2, 4],
       [1, 3, 5]])
```



12.5 ndarray의 재구성^{reshape}



LAB 12-5: 배열의 재구성

1. `arange()` 함수를 사용하여 1에서 12까지의 원소를 가지는 1차원 배열 `a1`을 생성하여라. 그리고 이 `a1` 배열을 `reshape()` 메소드를 사용하여 2행 6열의 행렬로 재구성하여라.

```
a1 = [[ 1 2 63 4 5 6]
      [ 7 8 9 10 11 12]]
```

2. `arange()` 함수를 사용하여 1에서 30까지의 원소를 가지는 1차원 배열 `a2`를 생성하여라. 그리고 이 `a2` 배열을 `reshape()` 메소드를 사용하여 3행 10열의 행렬로 재구성하여라.

```
a2 = [[ 1 2 3 4 5 6 7 8 9 10]
      [11 12 13 14 15 16 17 18 19 20]
      [21 22 23 24 25 26 27 28 29 30]]
```



12.5 ndarray의 재구성reshape

3. 문제 2에서 생성한 a2 배열을 reshape() 메소드를 사용하여 6행 5열의 행렬로 재구성한 행렬 a3를 생성하여라.

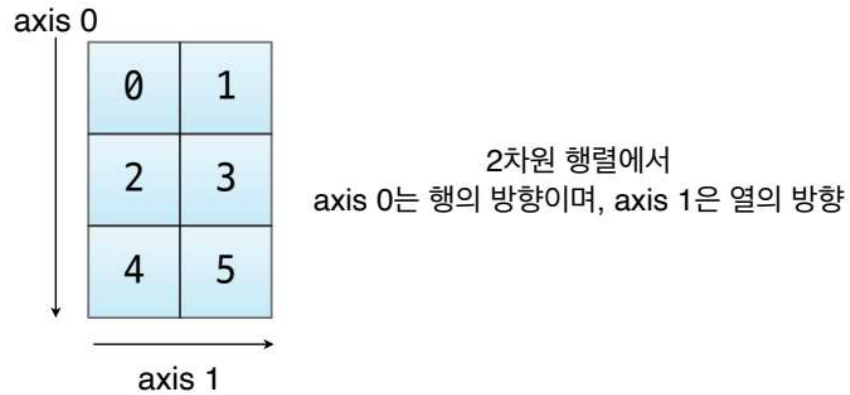
```
a3 = [[ 1 2 3 4 5]
      [ 6 7 8 9 10]
      [11 12 13 14 15]
      [16 17 18 19 20]
      [21 22 23 24 25]
      [26 27 28 29 30]]
```

4. 문제 3에서 생성한 a3 배열을 transpose() 함수를 사용하여 다음과 같은 전치 행렬을 생성하여라.

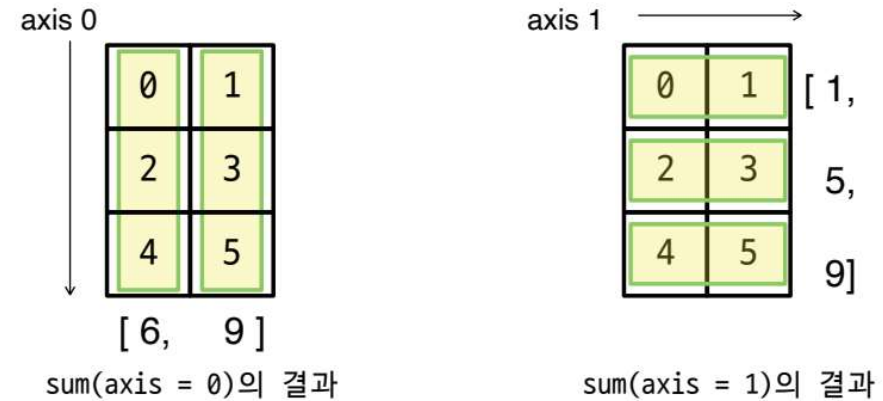
```
a4 = [[ 1 6 11 16 21 26]
      [ 2 7 12 17 22 27]
      [ 3 8 13 18 23 28]
      [ 4 9 14 19 24 29]
      [ 5 10 15 20 25 30]]
```



12.6 다차원 배열의 축



[그림 12-10] 2차원 배열과 axis 0과 axis 1의 방향



[그림 12-11] axis 0과 axis 1에 대하여 각각 sum() 함수를 수행한 결과



12.6 다차원 배열의 축

대화창 실습: sum() 함수와 axis에 따른 원소의 합

```
>>> a = np.arange(0, 6).reshape(3, 2)
>>> a
array([[0, 1],
       [2, 3],
       [4, 5]])
>>> a.sum()                # 행렬의 모든 원소의 합
15
>>> a.sum(axis = 0)        # 0축 방향(행 방향) 원소의 합
array([6, 9])
>>> a.sum(axis = 1)        # 1축 방향(열 방향) 원소의 합
array([1, 5, 9])
```

대화창 실습: 0축과 1축 방향 원소의 최솟값, 최댓값

```
>>> a.min(axis = 0)        # 0축 방향 원소의 최솟값
array([0, 1])
>>> a.min(axis = 1)        # 1축 방향 원소의 최솟값
array([0, 2, 4])
>>> a.max(axis = 0)        # 0축 방향 원소의 최댓값
array([4, 5])
>>> a.max(axis = 1)        # 1축 방향 원소의 최댓값
array([1, 3, 5])
```

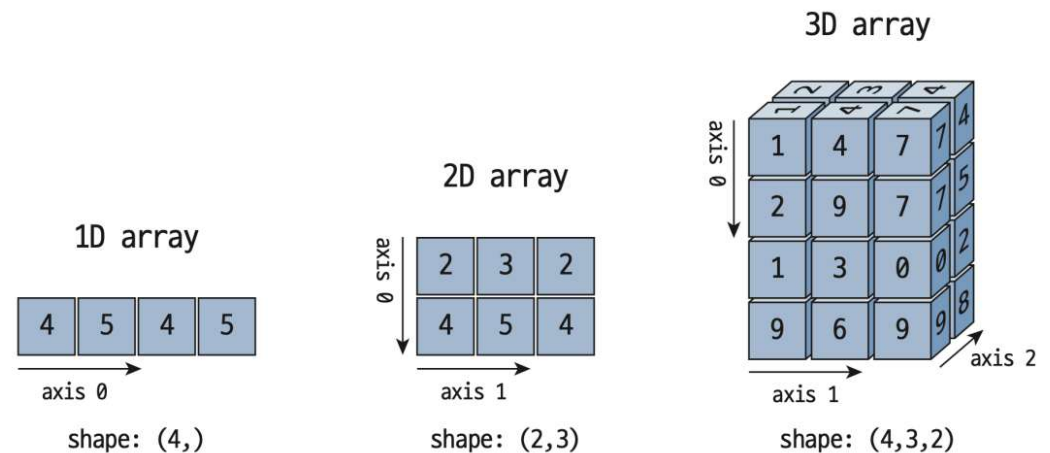


12.6 다차원 배열의 축



대화창 실습: ndarray의 insert() 함수

```
>>> a = np.array([1, 3, 4])
>>> np.insert(a, 1, 2)
array([1, 2, 3, 4])
>>> a = np.array([[1, 1], [2, 2], [3, 3]])
>>> np.insert(a, 1, 4, axis = 0)
array([[1, 1],
       [4, 4],
       [2, 2],
       [3, 3]])
>>> np.insert(a, 1, 4, axis = 1)
array([[1, 4, 1],
       [2, 4, 2],
       [3, 4, 3]])
```



[그림 12-12] 1차원, 2차원, 3차원 배열의 형태와 각 배열의 축 방향



12.6 다차원 배열의 축



LAB 12-6: 다차원 배열의 축과 삽입 함수

1. 1에서 100까지의 랜덤 정수 15개를 `np.random.randint()`를 통해서 생성하여라. 이렇게 생성한 정수값들을 (3, 5)의 shape를 가지는 행렬 `a`에 다음과 같이 저장하여 출력하여라.

```
a = [[82 30 87 64 21]
      [60 10 26 29 87]
      [29 99 37 21 96]]
```

2. 문제 1에서 생성한 `a` 행렬에 대해 열방향의 최댓값, 최솟값, 평균을 각각 다음과 같이 출력하시오.

```
a의 열방향 최댓값 : [82 99 87 64 96]
a의 열방향 최솟값 : [29 10 26 21 21]
a의 열방향 평균 : [57. 46.33333333 50. 38. 68. ]
```

3. 문제 1에서 생성한 `a` 행렬에 대해 행방향의 최댓값, 최솟값, 평균을 각각 다음과 같이 출력하시오.

```
a의 행방향 최댓값 : [87 87 99]
a의 행방향 최솟값 : [21 10 21]
a의 행방향 평균 : [56.8 42.4 56.4]
```




12.7 배열의 인덱싱과 슬라이싱



대화창 실습: 1차원 배열 인덱싱하기

```
>>> a = np.array([1, 2, 3])
>>> print(a[0], a[1], a[2])
1 2 3
>>> print(a[-1], a[-2], a[-3])
3 2 1
```



대화창 실습: 리스트와 슬라이싱

```
>>> a_list = [10, 20, 30, 40, 50, 60, 70, 80]
>>> a_list[1:5]
[20, 30, 40, 50]
```



대화창 실습: 1차원 배열에서 여러 개의 원소를 인덱싱하기

```
>>> a = np.array([1, 2, 3, 4, 5])
>>> print(a[np.array([0, 1])])
[1 2]
>>> print(a[np.array([0, 1, 2])])
[1 2 3]
>>> print(a[np.array([0, 1, 3])])
[1 2 4]
>>> print(a[np.array([1, 1, 1, 1])])
[2 2 2 2]
```




12.7 배열의 인덱싱과 슬라이싱



대화창 실습: 넘파이의 슬라이싱

```
>>> a = np.array([10, 20, 30, 40, 50, 60, 70, 80])
>>> a[1:5]      # 슬라이싱 구간 [시작:끝] 인덱스
array([20, 30, 40, 50])
>>> a[1:]
array([20, 30, 40, 50, 60, 70, 80])
>>> a[:]        # 전체를 슬라이싱
array([10, 20, 30, 40, 50, 60, 70, 80])
>>> a[::2]      # 양수 2의 스텝값
array([10, 30, 50, 70])
>>> a[::-1]     # 음수 스텝값
array([80, 70, 60, 50, 40, 30, 20, 10])
```



12.7 배열의 인덱싱과 슬라이싱



LAB 12-7: 배열의 인덱싱과 슬라이싱

1. 1에서 10까지의 원소를 가지는 1차원 배열 `a`를 생성하여라. 이 `a`를 인덱싱하여 `[2, 4, 6, 8]`의 원소를 가진 배열 `b`를 생성하여 다음과 같이 출력하여라.

```
a = [1 2 3 4 5 6 7 8 9 10]
b = [2 4 6 8]
```

2. 문제 1에서 생성한 배열 `a`를 슬라이싱하여 다음과 같은 배열 `b`, `c`, `d`, `e`, `f`를 생성하여라.

```
b = [6 7 8 9 10]
c = [7 8 9 10]
d = [1 2 3]
e = [1 3 5 7 9]
f = [10 8 6 4 2]
```



12.8 고차원 배열의 인덱싱

행 방향 인덱스 i

0	1
2	3
4	5

열 방향 인덱스 j

a[0, 0]	a[0, 1]
a[1, 0]	a[1, 1]
a[2, 0]	a[2, 1]

a[i, j] 인덱스의 위치

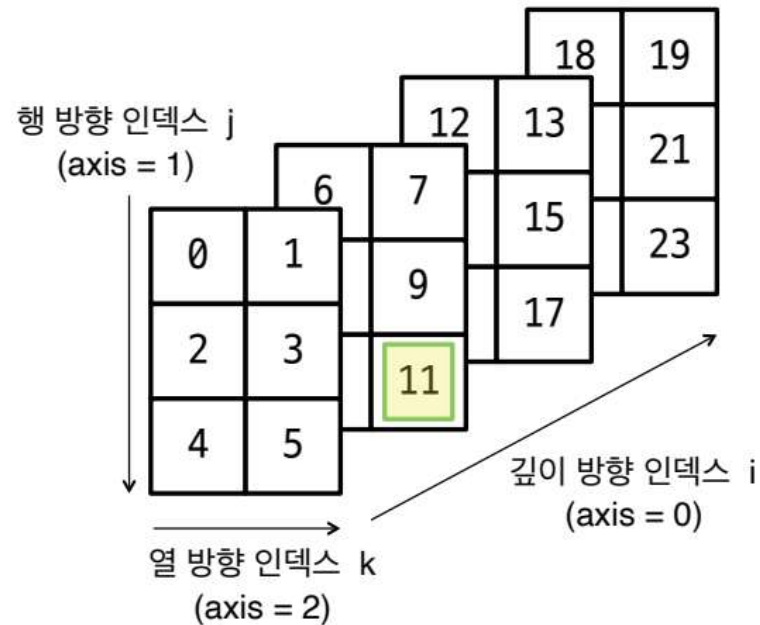
[그림 12-13] 2차원 배열의 인덱싱

대화창 실습: 2차원 배열의 인덱싱

```
>>> a = np.arange(0, 6).reshape(3, 2)
>>> a
array([[0, 1],
       [2, 3],
       [4, 5]])
>>> print(a[0, 0])
0
>>> print(a[0, 1])
1
>>> print(a[0, 2])
...
IndexError: index 2 is out of bounds for axis 1 with size 2
```



12.8 고차원 배열의 인덱싱



(4, 3, 2) 형태의 3차원 배열과 인덱싱 순서

[그림 12-14] 3차원 배열의 인덱싱

대화창 실습: 3차원 배열의 인덱싱

```
>>> a = np.arange(0, 24).reshape(4, 3, 2)
>>> a
array([[[ 0, 1],
        [ 2, 3],
        [ 4, 5]],
       [[ 6, 7],
        [ 8, 9],
        [10, 11]],
       [[12, 13],
        [14, 15],
        [16, 17]],
       [[18, 19],
        [20, 21],
        [22, 23]]])
>>> print(a[1, 2, 1])
11
```



12.8 고차원 배열의 인덱싱

Welcome to Python



대화창 실습: 3차원 배열의 인덱스와 concatenate() 함수

```
>>> a = np.arange(0, 24).reshape(4, 3, 2)
>>> a[0]
array([[ 0, 1],
       [ 2, 3],
       [ 4, 5]])
>>> a[0, 0]
array([ 0, 1])
>>> a[0, 1]
array([ 2, 3])
>>> a[0, 2]
array([ 4, 5])
>>> np.concatenate((a[0, 0], a[0, 2]), axis = 0)
array([0, 1, 4, 5])
```



12.9 2차원 배열의 슬라이싱



```
>>> a = np.arange(0, 9).reshape(3, 3)
>>> print(a[0]) # print를 이용한 출력 시 array()는 나타나지 않음
[0 1 2]
>>> print(a[0, :])
[0 1 2]
>>> print(a[:, 0])
[0 3 6]
```



12.9 2차원 배열의 슬라이싱

- 첫 번째 행의 두 성분 읽어오기

```
>>> print(a[0, 0:2])  
[0 1]  
>>> print(a[0, :2])  
[0 1]
```

- 2 x 2 배열 얻기

```
>>> print(a[0:2, 0:2])  
[[0 1]  
 [3 4]]  
>>> print(a[:2, :2])  
[[0 1]  
 [3 4]]
```




12.9 2차원 배열의 슬라이싱



```
>>> print(a[1:, 1:])  
[[4 5]  
 [7 8]]
```

```
>>> print(a[1, 1:])  
[4 5]
```

```
>>> a[1, 1:].shape  
(2,)  
>>> a[1:2, 1:].shape  
(1, 2)
```



12.9 2차원 배열의 슬라이싱

0	1	2
3	4	5
6	7	8

a[0]
a[0, :]

0	1	2
3	4	5
6	7	8

a[0, 0:2]

0	1	2
3	4	5
6	7	8

a[:2, :2]

0	1	2
3	4	5
6	7	8

a[1:, 1:]

0	1	2
3	4	5
6	7	8

a[1, 1:]

0	1	2
3	4	5
6	7	8

a[1, 1:]

0	1	2
3	4	5
6	7	8

a[1:2, 1:]

[그림 12-15] 2차원 배열의 슬라이싱과 구간 값



12.9 2차원 배열의 슬라이싱



LAB 12-8: 2차원 배열의 슬라이싱

1. 아래 그림과 같이 0에서 15까지의 연속적인 값을 원소로 가지는 4x4 크기의 2차원 배열 **a**를 생성하여라. 이 **a**에 대하여 인덱싱과 슬라이싱을 적용하여 다음과 같은 **b**, **c**, **d**, **e** 배열을 구하여라.

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

b

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

c

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

d

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

e

```
b = [ 1 5 9 13]
c = [ 9 10 11]
d = [[ 0 1] [ 4 5]]
e = [[ 5 6] [ 9 10]]
```

2. 문제 1에서 생성한 배열 **a**를 슬라이싱하고 평탄화 연산을 하여 다음과 같은 배열 **f**, **g**, **h**를 생성하여라.

```
f = [ 0 1 2 4 5 6]
g = [ 8 9 10 11 12 13 14 15]
h = [ 5 6 7 13 14 15]
```



12.10 선형 방정식 풀이와 행렬식



$$2x + 3y = 1$$

$$x - 2y = 4$$

[수식 12-1] 선형 연립 방정식



코드 12-1: 선형 연립 방정식 풀이
numpy_linear_ex.py

```
import numpy as np

a = np.array([[2, 3], [1, -2]])
b = np.array([1, 4])
x = np.linalg.solve(a, b)
print(x)
```

실행결과

```
[ 2. -1.]
```



12.10 선형 방정식 풀이와 행렬식

- 2×2 행렬의 행렬식: $\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$
- 3×3 행렬의 행렬식: $\det \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} = aei + bgf + cdh - ceg - bdi - afh$

[수식 12-2] 2×2 행렬의 행렬식과 3×3 행렬의 행렬식



대화창 실습: linalg.det() 함수를 사용한 행렬식

```
>>> a = np.array([[1, 2], [3, 4]])
>>> np.linalg.det(a)
-2.0000000000000004
>>> b = np.array([[1, 2], [3, -6]])
>>> np.linalg.det(b)
-12.0
>>> b = np.array([[1, 2], [1, 2]])
>>> np.linalg.det(b)
0.0
```



12.10 선형 방정식 풀이와 행렬식



LAB 12-9: 연립방정식 풀이

1. 다음과 같은 연립방정식의 해를 구하시오.

$$\begin{aligned}x + y - z &= 0 \\2x - y + 3z &= 9 \\x + 2y + z &= 8\end{aligned}$$

이 연립방정식의 해 x, y, z 를 다음과 같이 출력하시오.

```
x = 1.0, y = 2.0, z = 3.0
```

2. 1번 문제의 연립방정식은 다음과 같은 행렬 A 를 가진다.

```
A = np.array([[1, 1, -1], [2, -1, 3], [1, 2, 1]], dtype = 'int32')
```

이 행렬의 행렬식을 `linalg.det()` 함수를 사용하여 다음과 같이 구하여라.

3. 3×3 행렬 A 의 행렬식은 다음과 같이 정의할 수 있다. 문제 2번에 있는 A 행렬의 행렬식 -11이 아래 수식의 가장 오른쪽 항의 행렬식의 연산과 그 결과가 같음을 보이시오. 이를 위하여 3개의 2×2 행렬의 행렬식과 각각의 행렬식에 $a, -b, c$ 를 곱한 값을 더하시오.

$$|A| = \begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = a \begin{vmatrix} \square & \square & \square \\ \square & e & f \\ \square & h & i \end{vmatrix} - b \begin{vmatrix} \square & \square & \square \\ d & \square & f \\ g & \square & i \end{vmatrix} - c \begin{vmatrix} \square & \square & \square \\ d & e & \square \\ g & h & \square \end{vmatrix} = a \begin{vmatrix} e & f \\ h & i \end{vmatrix} - b \begin{vmatrix} d & f \\ g & i \end{vmatrix} + c \begin{vmatrix} d & e \\ g & h \end{vmatrix}$$



Welcome
to
Python

Questions?